

과제 절차서

LLM 할루시네이션 탐지 시스템

- 프로젝트명: LLM 할루시네이션 탐지 MCP 서버
- GitHub 주소:
https://github.com/Gachon-Unv-AIProgramming-Group-C/AI_Programming_Group_C_2026
- 작성일: 2026-06-19
- 작성자: Group C

1. 시스템 개요

본 시스템은 LLM 응답에서 사실 오류(할루시네이션)를 자동 탐지하는 MCP 도구 서버이다.

참고 논문: "Verify when Uncertain: Beyond Self-Consistency in Black Box Hallucination Detection" (arXiv:2502.15845, 2025)

탐지는 아래 3계층 알고리즘으로 수행된다.

- Layer 1 (LSC): 로그 확률 기반 신뢰도 측정
- Layer 2 (SINdex): LLM 샘플링 및 의미 클러스터링 기반 불일치 측정
- Layer 3 (SAC3): 패러프레이즈 기반 교차 모델 일관성 검증

2. 연구에 사용된 데이터

본 시스템은 아래 두 개의 공개 데이터셋으로 모델을 학습하였다. 데이터는 학습 스크립트 실행 시 인터넷에서 자동 다운로드되며, 별도 파일 첨부는 불필요하다.

KLUE-NLI v1.1

- 용도: NLI 분류 모델 파인튜닝 (train 24,998쌍 / val 3,000쌍)
- 출처: <https://github.com/KLUE-benchmark/KLUE>

KakaoBrain ParaKQC v1

- 용도: 패러프레이저 SFT 학습 (5,000쌍)
- 출처: <https://github.com/warnikchow/paraKQC>

학습된 모델은 HuggingFace Hub에 공개되어 있다.

- NLI 모델: <https://huggingface.co/serize/klue-roberta-base-nli-stage2>
- 패러프레이저: <https://huggingface.co/serize/local-qwen-paraphraser>

3. 환경 요구사항

하드웨어

- RAM: 16GB 이상
- GPU: NVIDIA GPU (VRAM 6GB 이상) 권장. 없을 경우 CPU 모드로 동작하나 속도가 매우 느림.
- 저장공간: 20GB 이상 여유 필요
- 인터넷: 최초 모델 다운로드 시 필수

소프트웨어 사전 설치

아래 항목이 모두 설치되어 있어야 검증을 진행할 수 있다.

소프트웨어	요구 버전	확인 명령어
Git	2.x 이상	`git --version`
Node.js	20.x 이상 (LTS)	`node --version`
Python	3.11.x	`python --version`
CUDA Toolkit	12.4 (GPU 보유 시)	`nvidia-smi`
Claude Code CLI	최신	`claude --version`

Claude Code CLI 설치:

```
npm install -g @anthropic-ai/claude-code
```

계정 준비

HuggingFace Read 토큰이 필요하다. <https://huggingface.co/settings/tokens> 에서 발급한다. (Read 권한이면 충분)

4. 검증 절차

절차 1 — 소스코드 클론

```
git clone https://github.com/Gachon-Unv-AIProgramming-Group-C/AI_Programming_Group_C_2026.git
cd AI_Programming_Group_C_2026/Project/Sever
```

확인: nli_server.py, package.json, .env.example 파일이 존재해야 한다.

절차 2 — 환경변수 설정

```
# Windows
copy .env.example .env
```

```
# Mac / Linux
cp .env.example .env
```

.env 파일을 열어 아래 내용으로 수정한다.

```
PORT=8000

# Python LLM
OPENAI_API_KEY=
ANTHROPIC_API_KEY=

# HuggingFace Read ( )
HF_API_KEY=hf_

# ID ( )
HF_MODEL_ID=jhgan/ko-sroberta-nli
HF_STAGE2_MODEL_ID=serize/klue-roberta-base-nli-stage2
HF_GEN_MODEL_ID=Qwen/Qwen2.5-3B-Instruct

NLI_SERVER_PORT=8001
LOG_LEVEL=info
```

확인: HF_API_KEY 값이 hf_ 로 시작하는 토큰으로 입력되어 있어야 한다.

절차 3 — Node.js 의존성 설치

```
npm install
```

확인: node_modules 폴더가 생성되어야 한다.

절차 4 — Python 가상환경 구성

가상환경 생성 및 활성화

```
# Windows
python -m venv .venv
.venv\Scripts\activate

# Mac / Linux
python3.11 -m venv .venv
source .venv/bin/activate
```

확인: 터미널 프롬프트 앞에 (.venv) 가 표시되어야 한다.

PyTorch 설치

```
# NVIDIA GPU (CUDA 12.4)
pip install torch --index-url https://download.pytorch.org/whl/cu124

# GPU (CPU )
```

```
pip install torch --index-url https://download.pytorch.org/whl/cpu
```

나머지 패키지 설치

```
pip install -r requirements-nli.txt
```

절차 5 — AI 모델 다운로드

최초 실행 시 약 6~10 GB 다운로드가 필요하다. 인터넷 속도에 따라 5~30분 소요된다.

```
python -c "  
from huggingface_hub import snapshot_download, login  
with open('.env') as f:  
    for line in f:  
        if line.startswith('HF_API_KEY='):  
            token = line.split('=', 1)[1].strip()  
            break  
login(token=token, add_to_git_credential=False)  
print('Downloading NLI model...')  
snapshot_download('serize/klue-roberta-base-nli-stage2', token=token)  
print('Downloading paraphraser model...')  
snapshot_download('serize/local-qwen-paraphraser', local_dir='./local-qwen-paraphraser', token=token)  
print('Downloading generation model (Qwen 3B)...')  
snapshot_download('Qwen/Qwen2.5-3B-Instruct')  
print('All models downloaded successfully!')  
"
```

확인: All models downloaded successfully! 메시지가 출력되어야 한다.

절차 6 — Python NLI 서버 실행

새 터미널 창을 열고 아래를 실행한다. 이 터미널은 계속 열어 두어야 한다.

```
cd AI_Programming_Group_C_2026/Project/Sever  
  
# ■■■■■ ■■■■  
.venv\Scripts\activate          # Windows  
source .venv/bin/activate       # Mac / Linux  
  
python nli_server.py
```

아래 로그가 모두 출력될 때까지 기다린다. 최대 3~5분 소요된다.

```
[nli_server] Using device: cuda  
[nli_server] NLI Model loaded.  
[nli_server] Paraphrase model loaded.  
[nli_server] General QA model loaded on GPU (4-bit quantized).  
[nli_server] Starting on port 8001  
* Running on http://127.0.0.1:8001
```

Using device: cpu 가 출력되는 경우는 GPU가 없거나 CUDA PyTorch가 설치되지 않은 것이다.
동작은 하나 속도가 매우 느리다.

헬스체크:

```
# Windows
Invoke-RestMethod http://127.0.0.1:8001/health

# Mac / Linux
curl http://127.0.0.1:8001/health
```

기대 응답: {"status": "ok", "nli_model": "serize/klue-roberta-base-nli-stage2", ...}

절차 7 — NestJS MCP 서버 실행

또 다른 새 터미널 창에서 실행한다.

```
cd AI_Programming_Group_C_2026/Project/Sever
npm run start:dev
```

확인: 아래 로그가 출력되어야 한다.

```
[Nest] LOG Server running at: http://localhost:8000
[Nest] LOG MCP endpoint: http://localhost:8000/mcp
```

핑 테스트:

```
# Windows
Invoke-RestMethod -Method Post -Uri http://localhost:8000/mcp -ContentType "application/json" -Body

# Mac / Linux
curl -X POST http://localhost:8000/mcp -H "Content-Type: application/json" -d '{"jsonrpc":"2.0","id"
```

기대 응답: {"jsonrpc":"2.0","id":1,"result":{}}

절차 8 — Claude MCP 등록

```
claude mcp add --transport http hallucination-detector http://localhost:8000/mcp
```

확인:

```
claude mcp list
```

출력에 hallucination-detector: http://localhost:8000/mcp (HTTP) - Connected 가 있어야 한다.

5. 기능 검증 테스트

두 서버가 모두 실행 중인 상태에서 claude 명령으로 세션을 시작한 뒤 아래 테스트 케이스를
순서대로 입력한다.

TC-01: 사실 부정 표현 탐지

입력:

Please use the check_hallucination tool to verify this:

Question: ■■■ ■■■ ■■■?

Response: ■■■ ■■■■.

합격 기준: verdict 가 HALLUCINATION, is_hallucination 이 true

TC-02: 역사적 날짜 오류 탐지

입력:

Please use the check_hallucination tool to verify this:

Question: ■■■■■ ■■■■■ ■■■ ■■■?

Response: ■■■■■ 1950■■ ■■■■■■.

합격 기준: verdict 가 HALLUCINATION, is_hallucination 이 true

TC-03: 과학적 사실 오류 탐지

입력:

Please use the check_hallucination tool to verify this:

Question: ■■ ■■■■ ■ ■■■? (1■■ ■■)

Response: ■■ 80■■■ ■■■■.

합격 기준: verdict 가 HALLUCINATION, is_hallucination 이 true

TC-04: 제품 제조사 오류 탐지

입력:

Please use the check_hallucination tool to verify this:

Question: ■■■■ ■■ ■■■ ■■■?

Response: ■■■■ ■■■■■ ■■■ ■■■■■■.

합격 기준: verdict 가 HALLUCINATION, is_hallucination 이 true

TC-05: 영어 저자 정보 오류 탐지

입력:

Please use the check_hallucination tool to verify this:

Question: Who wrote the play "Romeo and Juliet"?
Response: Romeo and Juliet was written by Charles Dickens in 1597.

합격 기준: verdict 가 HALLUCINATION, is_hallucination 이 true

6. 합격 판정 기준

- Python NLI 서버: /health 응답이 {"status":"ok"} 를 포함해야 한다.
- NestJS MCP 서버: ping 응답이 {"result":{}} 를 포함해야 한다.
- Claude 연동: claude mcp list 에서 Connected 상태여야 한다.
- 테스트 케이스: TC-01 ~ TC-05 중 4개 이상 HALLUCINATION 판정이어야 한다.

7. 예상 오류 및 대처

`ModuleNotFoundError: No module named 'flask'`

가상환경이 활성화되지 않은 상태이다. .venv\Scripts\activate 를 먼저 실행한다.

서버 기동 후 오랫동안 로그가 없는 경우

Qwen 3B 모델 로딩 중으로 정상이다. Starting on port 8001 로그가 출력될 때까지 기다린다.

`Port 8000 already in use`

포트를 점유 중인 프로세스를 종료해야 한다.

```
# Windows
netstat -ano | findstr :8000
taskkill /PID <PID> /F
```

테스트 결과가 UNCERTAIN으로 나오는 경우

LLM 샘플링의 무작위성에 의한 간헐적 현상이다. 동일 케이스를 재실행하면 대부분 HALLUCINATION 판정이 나온다.

HuggingFace 인증 오류 (401)

.env 파일의 HF_API_KEY 값이 잘못된 것이다. 토큰을 재발급 후 수정한다.

8. 예상 소요 시간

단계	소요 시간
사전 소프트웨어 설치	30~60분
클론 및 npm install	5분
Python 환경 구성 및 모델 다운로드	10~30분 (최초 1회)

단계	소요 시간
서버 기동	3~5분
테스트 케이스 5개 실행	25~40분
총계	약 1~2시간 (최초 기준)

모델이 이미 다운로드된 상태라면 서버 기동 및 테스트만 30~40분 내 완료 가능하다.